

rvLLM: Measured Gemma 4 Runtime Throughput in Rust on NVIDIA H100

Andy Norris (m0at), solidSF, Inc.

<https://github.com/solidSF-inc/rvLLM>

San Francisco, April 19, 2026; revised July 14, 2026

Abstract

rvLLM is a Rust runtime for Gemma 4 inference. This revision reports a public rvLLM v0.3.0 H100 measurement against one model revision, one sm_90 release archive, and a public receipt set.

Measured H100 path. In a synthetic fixed-geometry forward sweep, the forced CUTLASS projection route reached a median 7,492 candidate outputs/s at batch 512; the forced cuBLASLt projection route reached 6,356, a 17.9% difference. This is an internal route comparison, not a comparison with another serving engine.

Measurement boundary. The timed loop starts at the transformer layers, includes the language-model head and argmax, and reuses a persistent residual buffer initialized to zero once before warmup. Decode geometry and metadata remain fixed. It does not embed input tokens, feed the selected token back, advance position or context, or generate a valid sequence.

Design and validation. The path uses FP8 E4M3 weights, Hopper-specific projection kernels, FlashAttention-3 on sliding-attention layers, fused operations, CUDA graph replay, and batching. The receipts do not isolate the contribution of each mechanism. Correctness evidence is limited to published release-gate summaries for build, parity, ABI, sanitizer, archive, and server smoke.

1 Scope

rvLLM v0.3.0 [4] is an accelerator-oriented inference runtime with a native Rust serving path. Its public validation target is one NVIDIA H100 SXM 80 GB at sm_90. Metal and vision source are present but end-to-end generation and real-weight parity are not asserted for those paths. XLA/TPU work is experimental and no TPU server is included in this release.

This paper answers three narrow questions. First, what work does rvLLM execute in the measured fixed-geometry forward? Second, which design choices target efficient execution on Hopper? Third, what exactly was measured and checked in the v0.3.0 release? It deliberately does not claim model-quality improvement, end-to-end serving throughput, or superiority over another inference engine.

The April paper reported older GPU and TPU results. Those GPU rows were later withdrawn after revalidation found invalid KV-page and block-table geometry. This revision replaces them with the public v0.3.0 H100 receipt set and does not carry forward TPU performance claims that lack raw receipts and immutable model revisions.

2 Runtime design

The serving path starts in `rvllm-serve`, which validates the local HTTP request and submits bounded work. `rvllm-runtime` constructs the batch plan and executes the model step. Loader, metadata, and memory crates validate model tensors, layouts, and device storage; attention, fused-operation, CUTLASS, kernel, graph, and sampling crates own accelerator execution. Missing or incompatible accelerator artifacts are errors rather than silent fallbacks.

The model is loaded before generation. Prefill processes prompt tokens and populates the paged KV cache. Ordinary decode then advances one token per sequence per step; the optional speculative path verifies chunks of draft tokens. Each transformer layer applies normalization, Q/K/V projection, rotary position encoding and KV update, attention, output projection, feed-forward projection, activation, and residual updates. Final normalization, the language-model head, and sampling produce the next token.

Paged KV state removes the requirement that each sequence occupy its own contiguous KV region while using one preallocated device arena. Slot mappings, sequence lengths, block tables, and graph replay inputs must be refreshed together. This follows the general memory-management motivation of PagedAttention [1], while the public rvLLM implementation and its supported geometry are release-specific.

3 How rvLLM targets speed

3.1 Less weight traffic and Hopper-specific matrix work

The measured model stores linear weights in FP8 E4M3 and retains the KV cache in F16. Relative to BF16, FP8 reduces the bytes moved for the weight matrices and maps to Hopper tensor-core execution; the format and scaling tradeoffs are described by Micikevicius et al. [2]. rvLLM loads the model’s channel scales explicitly and exposes two internal linear-algebra routes: CUTLASS channel-scale kernels and cuBLASLt. CUTLASS provides the template machinery used to specialize matrix multiplication for the target architecture [6]. The forced route changes channel-scale transformer projection GEMMs. The language-model head remains on cuBLASLt, and the remaining forward work is shared.

The two routes do not have one universal winner. In the measured sweep, cuBLASLt leads at batches 1, 8, and 32; CUTLASS leads at batches 64 through 512. This crossover is the concrete reason to keep route choice explicit and shape-aware. The receipt does not show that an autotuned policy selected these winners in the measured forward path: the published policy is a deterministic baseline, and the channel-scale execution path does not consume its resolved plans. The paper therefore reports both forced routes rather than an invented “automatic winner” result.

3.2 Attention, fusion, and launch amortization

The H100 bundle uses FlashAttention-3 for sliding-attention layers; global attention layers use the validated `fa_sm89_*` fallback path. FlashAttention-3 uses Hopper asynchrony and low-precision support to improve attention execution efficiency [7]. rvLLM also fuses operations such as Q/K/V preparation, normalization and residual work where the release kernel inventory provides a fused entry point. These choices are intended to reduce intermediate traffic and the number of separately scheduled operations. The context length in the public throughput sweep is one token, however, so that sweep is not an attention-scaling experiment and cannot quantify the benefit of FlashAttention-3 at long context.

Every timed sample uses CUDA graph capture. A graph records a launch workflow for replay, reducing repeated host setup and exposing the workflow as a unit to the CUDA runtime [5]. The graph, model state, and device buffers are established before timing. The receipts do not include a graph-on versus graph-off ablation, so graph replay is a documented mechanism, not a separately measured percentage improvement.

Batching is the final visible lever. Each repeated fixed-geometry forward computes one argmax candidate per batch row. The benchmark records per-sample aggregate candidate rate as

$$T_i = \frac{1000B}{t_i}, \quad (1)$$

where B is batch size and t_i is sample i 's measured step time in milliseconds. The release summary reports the median of the resulting T_i values. Larger batches amortize fixed launch and weight-movement costs and expose more matrix parallelism. The observed rise from 46.7 to 7,492 candidate outputs/s on the CUTLASS route is consistent with that mechanism, but it is fixed-geometry aggregate throughput rather than sequence generation or per-request latency.

4 Measurement method

The receipt set is labeled July 13, 2026 and accompanies the v0.3.0 release. It reports the source, model, kernel bundle, driver, toolchain, and harness parameters shown in Table 1. The model is `RedHatAI/gemma-4-31B-it-FP8-Dynamic` at immutable revision beginning `448f20b6eb8b`. The model manifest records an aggregate SHA-256 beginning `31d8f68ca323`; the complete values are in the linked receipt set. The kernel archive and receipt bytes have published SHA-256 values. The source/model/environment associations are operator-attested: the receipts do not hash the benchmark executable, and the model manifest does not publish the file-list/hash procedure behind its aggregate.

Item	Reported value
rvLLM source	<code>afcad6c0145b4455ce91aca52d5a5de4eb4e1572</code>
Hardware	NVIDIA H100 SXM 80 GB HBM3, <code>sm_90</code>
Driver / CUDA	NVIDIA 555.58.02 / CUDA 12.4
Toolchain	<code>rustc 1.97.0</code> ; <code>nvcc 12.4</code>
CUTLASS / FA3	<code>da5e086dab31d638</code> / <code>1233b73b6c95340c</code>
Weight / KV dtype	FP8 E4M3 / F16
Release archive	<code>rvllm-kernels-sm90-afcad6c0145b.zip</code>
Archive SHA-256	<code>97a8a8de4218733514e493193c4f3182ff5b309bf9225fd135e9b64231e6d250</code>

Table 1: Reported environment and kernel release artifact. The kernel archive and receipt bytes are hash-identified; other associations are operator-attested [3].

The benchmark constructs a synthetic fixed-geometry state with one context position per batch row, block size 32, 1,024 total KV blocks, FP8 E4M3 weights, F16 KV state, and CUDA graph replay enabled. It zeroes the residual once before warmup, then reuses that persistent buffer while each repetition starts at the transformer layers and computes through the final language-model head and argmax without embedding a token or advancing input, position, context length, slot mapping, or block metadata. It measures batches 1, 8, 32, 64, 128, 256, and 512. Each sample performs 10 warmup iterations followed by 100 timed iterations. There are two valid samples per route and batch, except batch 512, which has three. All 30 samples are retained; none are discarded. The table reports the median recorded by the release summary. Because a two-sample median is only a midpoint, the small replicate count is a limitation rather than a broad statistical characterization.

The timer therefore covers a model-shaped fixed-geometry forward, not valid autoregressive generation. It does not cover model loading, token embedding, request parsing, tokenization, prompt prefill, HTTP transport, or sequence-state updates. The public data are project-produced receipts, not an independent reproduction. They also omit the arena size, executable hash, run order, clocks, and power state.

5 Results

Table 2 and Figure 1 report the complete H100 sweep. cuBLASLt leads at the three smallest batches, while CUTLASS leads from batch 64 onward. The largest measured median is 7,492.040 candidate outputs/s for CUTLASS at batch 512. At that same batch, cuBLASLt reaches 6,356.098 candidate outputs/s; the relative difference is 17.9%. This compares the two forced channel-scale projection routes under the same harness; the language-model head remains on cuBLASLt and the remaining work is shared. It is not an rvLLM-versus-vLLM result.

Batch	CUTLASS cand./s	cuBLASLt cand./s	Leading route
1	46.704	48.768	cuBLASLt
8	369.495	421.023	cuBLASLt
32	1,447.185	1,543.125	cuBLASLt
64	2,764.438	2,696.043	CUTLASS
128	5,048.066	4,405.914	CUTLASS
256	7,186.885	5,647.870	CUTLASS
512	7,492.040	6,356.098	CUTLASS

Table 2: Median aggregate candidate rate for the synthetic fixed-geometry forward. The release summary labels this quantity tokens/s.

The curve shows diminishing throughput gains after batch 256 on the CUTLASS route. That observation is specific to this model, target, context, and fixed geometry. It does not establish the latency or throughput of a production request mix.

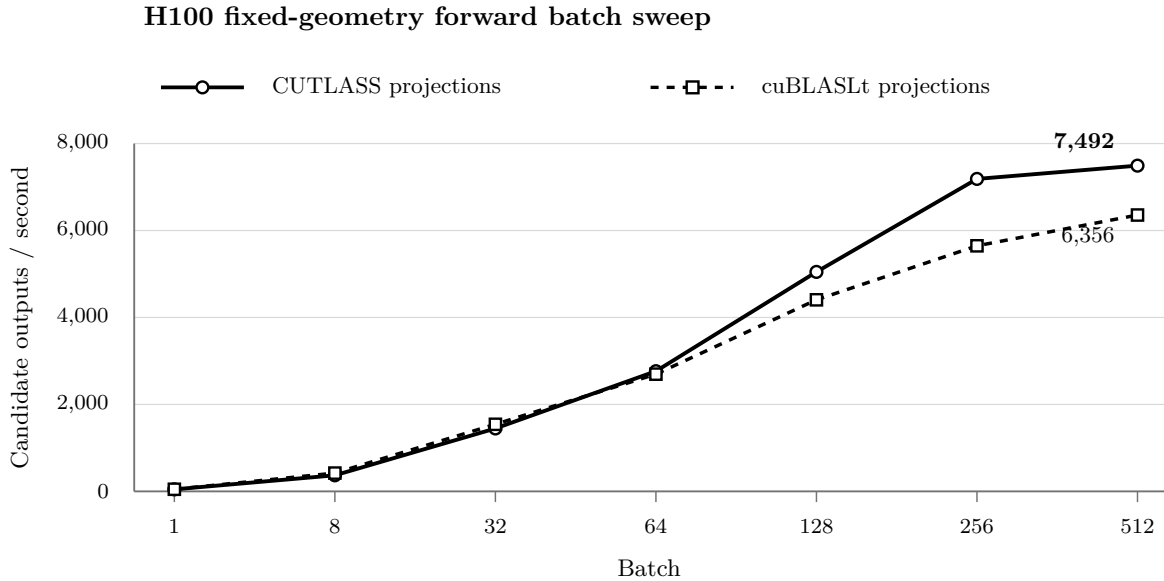


Figure 1: The seven-batch sweep from the public release summary. Markers show medians; whiskers span the full replicate range. There are two replicates per route and batch, except batch 512 with three. Higher aggregate candidate rate is better.

6 Correctness and release integrity

Performance evidence is more useful when release bytes are hash-identified and the execution path passes numerical and runtime checks. The v0.3.0 validation summary declares that the final kernel archive was re-extracted and rehashed, 54 files and 48 manifest artifacts were reverifed,

and deterministic archive reproduction passed. The source build gates report PASS for clean rebuild, FP8 decode-v2 parity, and FlashAttention-3 sm_90 parity.

The runtime ABI gate loaded 44 PTX files and resolved 90 PTX entries. Four shared libraries loaded with 25 expected symbols, four forbidden symbols were absent, and seven attention-contract values were checked. Compute Sanitizer reported zero errors and zero leaked allocations. These numbers describe the published gate result; the public summary and checksums are available, while the underlying tool transcript is not presented as an independent audit.

The exact-ZIP server-smoke summary reports ten passing checks: health, authenticated status, model listing, one completion token, one chat-completion token, metrics, authentication rejection, and rejection of unsupported streaming and stop requests. The server was observed bound to loopback. The implemented surface is `/health`, `/status`, `/metrics`, `/v1/models`, `/v1/completions`, and `/v1/chat/completions`. Generation is non-streaming, only one choice is accepted, and no Responses endpoint is implemented.

The release semantic audit reviewed 54 package files and scanned 84,504 text lines. It reports zero credential or secret hits, zero customer or production infrastructure hits, and zero private-path or solidSF-private-identifier hits. This is a release-boundary check, not a claim that automated scanning can prove the absence of every possible disclosure.

7 Limitations

The headline number is deliberately narrow. It is aggregate candidate rate from a synthetic fixed-geometry forward with context-length metadata set to one. No token embedding occurs. A residual buffer is zeroed once before warmup and then persists through replay while geometry and metadata remain unchanged; argmax is not fed back and no valid sequence is generated. The measurement excludes prefill, networking, request scheduling, and every sequence-state update. It measures one model revision on one self-reported H100 SXM 80 GB configuration and does not include a graph ablation, kernel-fusion ablation, power measurement, latency distribution, or external engine baseline. It also does not evaluate semantic quality, accuracy, perplexity, or safety of the model output.

The route sweep measures forced CUTLASS and cuBLASLt channel-scale projection paths. Although a policy artifact is structurally authenticated and its entries can be launched, the receipt explicitly states that the current channel-scale Gemma 4 path does not consume the resolved policy plans. Consequently, this paper makes no claim of autotuned winner selection in the measured execution.

The retained TPU snapshot in the project benchmark page has no raw receipts or immutable model revisions and is not used here. Metal, vision, and XLA/TPU source remain outside the end-to-end release claim. Additional measurements should cover real prompt distributions, growing contexts, time to first token, inter-token latency, sustained multi-request serving, and independent reproduction.

8 Conclusion

The measured H100 path combines FP8 weights, Hopper-specific projection kernels, sliding-layer FlashAttention-3, fused operations, CUDA graphs, and batching. The synthetic fixed-geometry harness reports 7,492 median candidate outputs/s for forced CUTLASS projections at batch 512, 17.9% above the forced cuBLASLt projection route. No ablation assigns causality, and no valid sequence is generated. Public hashes cover the kernel archive and receipts; other benchmark associations are operator-attested. This does not establish autoregressive or end-to-end throughput, model quality, or cross-engine superiority.

References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. doi: 10.1145/3600006.3613165.
- [2] Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.
- [3] Andy Norris. rvLLM H100 release validation receipts. Immutable public receipt set, 2026.
- [4] Andy Norris. rvLLM v0.3.0. GitHub release v0.3.0, 2026.
- [5] NVIDIA Corporation. CUDA graphs. CUDA Programming Guide, 2026. Accessed July 14, 2026.
- [6] NVIDIA Corporation. CUTLASS: CUDA templates for linear algebra subroutines. Git revision da5e086dab31d638, 2026.
- [7] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024.